FRONTEND OPTIMIZATION TECHNIQUES

@DimpleKumari

Forming a network of fantastic coders.

Frontend performance optimization is often limited to reducing JavaScript bundle size, image compression, and lazy loading. However, there are many lesser-known techniques that can significantly enhance page load times, reduce rendering overhead, and improve the user experience.

This document explores rare and underutilized frontend optimizations that can make a huge difference in performance but are not widely known among developers.

@DimpleKumari

CSS CONTAINMENT (CONTAIN)

What's the Problem?

Every time an element updates, the browser checks surrounding elements to see if they need re-rendering. This causes unnecessary recalculations, repaints, and reflows, slowing down performance.

Solution: Use contain Property

The contain CSS property isolates elements from the rest of the page, preventing unnecessary rendering updates.

Best for: Infinite scrolling lists, dynamic dashboards, and frequently updating UI components.

@DimpleKumari

Forming a network of fantastic coders.

CSS CONTAINMENT (CONTAIN)

How It Works:

- layout → The element's size and position won't affect other elements.
- paint → The browser doesn't repaint anything outside this element's boundaries.

```
.card {
  contain: layout paint;
}
```

@DimpleKumari

Forming a network of fantastic coders.

PARSING & PRELOADING

What's the Problem?

When a browser downloads HTML, it usually loads assets in sequence. This slows down page rendering.

Solution: Help the Browser Prioritize Assets

Browsers scan ahead for resources before execution. We can take advantage of this by manually preloading important assets.

```
<link rel="preload" href="styles.css" as="style">
<link rel="preload" href="script.js" as="script">
```

@DimpleKumari

Forming a network of fantastic coders.

STYLE QUERIES

What's the Problem?

Media queries depend on the viewport size, making them less effective for reusable components.

Solution: Use CSS Container Queries (@container)

Now, components can resize dynamically based on their own container, not the whole screen.

```
@container (min-width: 300px) {
  .button {
    font-size: 1.2rem;
  }
}
```

@DimpleKumari

Forming a network of fantastic coders.

ASYNC IMAGE DECODING (DECODING="ASYNC")

What's the Problem?

Images can block rendering because they need to be decoded before being displayed.

Solution: Use decoding="async" to Prioritize Page Content

This prevents the browser from blocking layout rendering while decoding images.

```
<img src="image.jpg" decoding="async" />
```

@DimpleKumari

Forming a network of fantastic coders.

CONTENT-VISIBILITY

What's the Problem?

The browser renders all elements on the page—even if they are not currently visible. This slows down performance.

Solution: Use content-visibility to Skip Offscreen Rendering

Use content-visibility to Skip Offscreen Rendering

```
.lazy-loaded {
  content-visibility: auto;
}
```

@DimpleKumari

Forming a network of fantastic coders.

WEBP & AVIF – NEXT-GEN IMAGE FORMATS FOR FASTER LOADING

What's the Problem?

Traditional image formats (PNG, JPEG) are large and slow to load.

Solution: Use WebP or AVIF for Faster Loading

These formats compress images without losing quality.

```
<picture>
  <source srcset="image.avif" type="image/avif">
  <source srcset="image.webp" type="image/webp">
  <img src="image.jpg" alt="Optimized Image">
</picture>
```

@DimpleKumari

Forming a network of fantastic coders.

THANK YOU FOR READING!

If you found this informative and valuable, I'd love for you to connect with me. Follow me Medium, Codepen, and connect with me on LinkedIn to stay updated on the latest in web development, interviews, and more.

Let's connect!

💼 **LinkedIn** — https://www.linkedin.com/in/dimple-kumari/

🔗 **Medium** — https://medium.com/@dimplekumari0228

✍️ **Codepen** — https://codepen.io/DIMPLE2802

@DimpleKumari

Forming a network of fantastic coders.

DIMPLE KUMARI

Forming a network of fantastic coders.